BCSE498J- Project - II

COMPARATIVE ANALYSIS OF REINFORCEMENT LEARNING ALGORITHMS FOR INTELLIGENT HOSPITAL SCHEDULING WITH FAIRNESS, EXPLAINABILITY, AND REAL-TIME DECISION SUPPORT DASHBOARD

Submitted in partial fulfilment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

22BCE2438 SHASHWAT CHAUDHARY

Under the Supervision of

Prof. Sunil Kumar

Assistant Professor Grade 1

School of Computer Science and Engineering

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

April, 2026

DECLARATION

I hereby declare that the project entitled "COMPARATIVE ANALYSIS OF REINFORCEMENT LEARNING ALGORITHMS FOR INTELLIGENT HOSPITAL SCHEDULING WITH FAIRNESS, EXPLAINABILITY, AND REAL-TIME DECISION SUPPORT DASHBOARD" submitted by me, for the award of the degree of Bachelor of Technology in Computer Science and Engineering to VIT is a record of bonafide work carried out by me under the supervision of Prof. Sunil Kumar, School of Computer Science and Engineering.

I further declare that the work reported in this project report has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore **SHASHWAT CHAUDHARY (22BCE2438)**

Date :

CERTIFICATE

This is to certify that the project entitled "COMPARATIVE ANALYSIS OF REINFORCEMENT LEARNING ALGORITHMS FOR INTELLIGENT HOSPITAL SCHEDULING WITH FAIRNESS, EXPLAINABILITY, AND REAL-TIME DECISION SUPPORT DASHBOARD" submitted by SHASHWAT CHAUDHARY (22BCE2438), School of Computer Science and Engineering, VIT, for the award of the degree of Bachelor of Technology in Computer Science and Engineering , is a record of bonafide work carried out by the student under my supervision during Winter Semester 2025-2026, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore **Signature of the Guide**

Date :

Internal Examiner **External Examiner**

Head of the Department

ACKNOWLEDGEMENT

I SHASHWAT CHAUDHARY (22BCE2438) am deeply grateful to the management of Vellore Institute of Technology (VIT) for providing me with the opportunity and resources to undertake this project. Their commitment to fostering a conducive learning environment has been instrumental in my academic journey. The support and infrastructure provided by VIT have enabled me to explore and develop my ideas to their fullest potential.

My sincere thanks to Dr. Jaisankar N, the Dean of the School of Computer Science and Engineering, for constant support and encouragement. The Dean's leadership and vision have greatly inspired me to strive for excellence. The dedication to academic excellence and innovation has been a constant source of motivation.

I express my profound appreciation to Dr. Boominathan P, the Head of the Department of Software Systems for insightful guidance and continuous support. The expertise and advice have been crucial in shaping the direction of my project. The commitment to fostering a collaborative and supportive atmosphere has greatly enhanced my learning experience. The constructive feedback and encouragement have been invaluable in overcoming challenges and achieving my project goals.

I am immensely thankful to my project supervisor, Prof. Sunil Kumar, School of Computer Science and Engineering, for dedicated mentorship and invaluable feedback. The patience, knowledge, and encouragement have been pivotal in the successful completion of this project. The willingness to share expertise and provide thoughtful guidance has been instrumental in refining my ideas and methodologies. This support has not only contributed to the success of this project but has also enriched my overall academic experience.

I extend my heartfelt thanks to all for the guidance and support received throughout this endeavor.

TABLE OF CONTENTS

EXECUTIVE SUMMARY

Hospitals have to deal with competing demands. They need to treat patients fast use limited resources wisely and make sure no one gets left behind. Old scheduling systems, which usually rely on fixed rules or manual processes do okay when things are normal but struggle when it gets busy or unpredictable. This leads to waiting times wasted resources in some areas while others are overwhelmed and overall lower quality of care.

This project looks at a way of doing things. We created a scheduling system that uses a type of intelligence called Reinforcement Learning (RL) to decide how to allocate patients. Of following pre-programmed rules the system learns by trying different things seeing what happens and figuring out what works best. We tried out three algorithms: Q-learning, Deep Q-Network (DQN) and Double Deep Q-Network (DDQN). The system considers how urgent a patients treatment is, their priority and whats available in time when making scheduling decisions.

To train and test the system we created data that mimics real hospital operations. We built a custom simulation to model patients arriving, limited resources and scheduling challenges. We also made a dashboard using Streamlit that lets us easily track scheduling decisions see how well the system is doing and monitor performance metrics.

The results are promising. The RL approach does better than scheduling methods leading to shorter waits more efficient resource use and fairer treatment across patient groups. Among the three algorithms DDQN performed the best. While the system currently only works in an environment its designed to be scalable and could be used in real-world healthcare scheduling applications, in the future.

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CPU	Central Processing Unit
DDQN	Double Deep Q-Network
DL	Deep Learning
DQN	Deep Q-Network
GPU	Graphics Processing Unit
ICU	Intensive Care Unit
MDP	Markov Decision Process
ML	Machine Learning
MSE	Mean Squared Error
NN	Neural Network
PPO	Proximal Policy Optimization
RAM	Random Access Memory
RL	Reinforcement Learning
UI	User Interface

SYMBOLS AND NOTATIONS

α	Learning rate
ε	Exploration rate (epsilon-greedy)
γ	Discount factor
π^*	Optimal policy
θ	Neural network weights (Q-network)
θ^-	Target network weights
a	Action taken by the agent
FP	Fairness penalty
G_t	Discounted cumulative return
$L(\theta)$	Loss function for neural network training
$Q(s,a)$	Action-value function (Q-function)
r	Immediate reward signal
RU	Resource utilization rate
s	Current state of the environment
s'	Next state after taking action
w_1,w_2,w_3	Reward function weight coefficients
WT	Patient waiting time

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Hospitals worldwide are constantly facing a very significant problem: How can they schedule their patients and at the same time allocate scarce resources such as doctors, beds and equipments, in a manner that besides being timely, it is also very effective? Intuitively, anyone to whom a hospital emergency room full of patients is not a stranger, would understand the problem. Hospital patients do not come in a predictable manner and their conditions can be of varying degrees of severity. On top of that the medical personnel and the infrastructure available in the hospital for their treatment are always limited. and the conventional method cannot manage it properly. Patients end up spending much more time waiting while resources are wasted. The quality of care is reduced.

During the last few years, Artificial Intelligence and Machine Learning have become powerful tools capable of solving numerous problems. For example, Reinforcement Learning techniques prove highly effective in making decisions in dynamic environments. A reinforcement learning agent learns through experience and makes decisions based on previous events. It does not have to rely on pre-established rules. This project proposes applying reinforcement learning to address the hospital scheduling issue. The agent will collect data on the patients' arrival rates, available resources, and urgency. Based on this data, it will make decisions concerning scheduling. The Deep Q-Network and Double Deep Q-Network will serve as decision-making tools.

Problem Domain Overview

This project focuses on devising a decision-making technique in a complex and changing environment. Reinforcement learning helps to determine the optimal strategy considering multiple factors such as urgency and resources. The system will be unique because it will adapt to changes in the conditions and learn from its own experiences.

Theoretical Foundations

This research is grounded in multiple disciplines. Reinforcement Learning provides the necessary framework, while Markov Decision Processes assist in modeling the problem. In case of complexity, deep learning may come to aid. The agent will

perceive the environment, make decisions, and then receive rewards or penalties. With time, it will develop a strategy to optimize performance and minimize losses.

Motivation

Hospitals require improved operations management, and a better scheduling system can increase patients' satisfaction and improve their health outcomes. Poor scheduling often leads to lengthy waiting lines, rising costs, and wasteful allocation of resources.

Healthcare organizations require solutions that can respond to dynamic conditions rapidly. We believe that Reinforcement Learning is an appropriate solution because it can learn from its experience.

From an academic perspective, this project is a unique opportunity to explore some fascinating technical issues. We will be able to compare different approaches in Reinforcement Learning.

On a technical level, we will have to devise a Reinforcement Learning environment and implement several algorithms. Additionally, we will build a dashboard to visualize the findings. The objective is to make our system efficient, equitable, and understandable.

Scope of the Project

This project aims to design a planning and scheduling system for hospitals based on Reinforcement Learning.

- **Functional Scope:**
  - Scheduling patients according to their priority and resource availability
  - Making decisions by applying Reinforcement Learning algorithms
  - Visualizing the findings with a dashboard
- **Non-Functional Scope:**
  - Ensuring scalability to handle many patients
  - Guaranteeing efficiency and performance
  - Improving maintainability and updating ability
- **Technical Scope:**

  - Using Python programming language and Reinforcement Learning libraries to design the system
  - Implementing multiple algorithms such as Q-Learning, Deep Q-Network, and Double Deep Q-Network
  - Integrating the algorithm with the dashboard for visualization

- **Project Boundaries**
  - Utilizing artificial data, not real data from hospitals
  - Not deploying the algorithm in a real hospital
  - Conducting evaluations within a simulation

PROPOSAL PROJECT DESCRIPTION AND GOALS

Literature Review

Researchers have been actively addressing the problem of patient scheduling and efficient resource utilization. For a long time, this area was focused on rule-based solutions, heuristic search and mathematical programming. These techniques proved to increase resource usage rate and decrease patient waiting times. However, in most of the cases, these systems have relied on assumptions about the stable environment and hence were rather susceptible to any deviations.

With the advent of machine learning and artificial intelligence, things changed. It became possible for machine learning models to detect complex relations in big volumes of data. The downside was that most of these approaches used single shot prediction models. Namely, the predictions were made based on available data without regard to changes in the dynamics. It was especially problematic when used in the hospital setting since patient flow and resource availability constantly changed. Therefore, reinforcement learning techniques became applicable in order to address sequential decision problems with changing conditions and aim to improve long-term outcomes.

More and more papers started using reinforcement learning in healthcare, including patient scheduling and treatment planning, bed allocation and staff scheduling, among others. In particular, the most popular approach involves modeling a scheduling problem as a Markov Decision Process with certain rewards and states representing current patient flow and available resources and with scheduling options being possible actions. However, traditional RL techniques, namely Q-learning, were shown to be limited to relatively small sets of actions and states, which is especially challenging to apply in reality.

It is deep reinforcement learning techniques that make the whole scheduling task manageable. Namely, DQN and DDQN models, among other approaches, combined RL with machine learning algorithms, specifically deep neural networks. In contrast to Q-learning algorithms, deep RL techniques do not rely on a giant Q-table, which is required to store the value for every state-action pair. Instead, neural network learns how to map values for actions based on the current state, thus being able to generalize to new situations.

It should also be acknowledged that there are still some disadvantages of the current models, which might limit their application in practice. First, all the techniques focus on increasing efficiency and neglect other aspects such as fairness of scheduling decisions. Also, deep RL models are usually a black box for users. Moreover, very few systems offer real-time dashboards to monitor scheduler's behavior. These weaknesses might be seen as gaps for further exploration.

Research Gap

Although there have been great improvements in this area recently, review of the literature has indicated several aspects hindering further development and practical implementation of these systems:

- Static Behavior: The traditional rule-based and mathematical optimization techniques are designed statically and do not change their strategy in face of unforeseen changes in the problem.
- Insufficient Deep RL Algorithms Usage: Some researches only use Q-learning approach despite numerous challenges associated with handling high-dimensional continuous space in hospital environment.
- Absence of Fairness Criteria: Very few systems try to tackle the issue of unfair distribution of treatment across patients of different priorities.
- Lack of Interpretation Capabilities: Decisions made by deep learning models are hard to track and explain, thus lacking transparency, which is critical in healthcare.
- Unfavorable Scalability Properties: While several approaches are promising in simulations, their performance significantly drops when applied to large cases of hospitals.
- Absence of Real-time Dashboards: Currently, almost none of the systems provides administrators with the tool to visualize the decisions and understand them.

Taking these drawbacks into account, it becomes apparent how necessary it is to create a more advanced scheduler system.

Objectives

Based on the identified weaknesses, the following objectives will be addressed by this project:

- To develop a reinforcement learning-based hospital scheduling system.
- To model a hospital scheduling scenario as a Markov Decision Process (MDP).
- To implement and evaluate Q-learning, DQN, and DDQN algorithms in order to analyze their differences and performance on scheduling problem.
- To minimize patient waiting times and to enhance resource utilization through reinforcement learning-based scheduling.
- To integrate fairness criterion into decision-making process in order to treat patients of all priority levels adequately.
- To create a convenient and informative user interface displaying scheduling process results.
- To conduct thorough evaluation of each technique in terms of multiple performance measures.

Problem Statement

Basically, the problem statement is as follows: the task is to schedule patients efficiently in a hospital setting given the fact that their resources are limited while patient arrivals are both dynamic and somewhat unpredictable. Consequently, traditional scheduling methods prove incapable of addressing these issues and cause a waste of patient waiting times and resource underutilization.

A possible solution to the problem is to implement a smart hospital scheduler, which would utilize reinforcement learning techniques in order to discover appropriate scheduling policies and ensure optimal resource utilization and fairness in decisions.

Project Plan

To accomplish this task, the project was broken down into six phases, each of which provided foundation for the next one. Dividing the project into phases allowed to maintain it under control and ensured proper development of every module.

- **Phase 1: Problem Definition and Data Preparation**

  At the beginning, it was necessary to thoroughly explore and analyze what hospital scheduling entails ” its restrictions, goals, and peculiarities. Due to the lack of authentic data from real hospitals (which makes sense anyway), synthetic patient flows were created with such attributes as time of arrival, priority levels, resources necessary, and more. All of these datasets served as the basis for later training.

- **Phase 2: Environment Design**

  Next, a custom simulation environment for hospitals wards was developed. Specifically, MDP was designed with state as a snapshot showing patients awaiting treatment and resource availability, actions representing possible schedules and rewards being associated with efficiency, fairness, and other criteria.

- **Phase 3: Model Implementation**

  Given that the environment had been created, three types of reinforcement learning techniques were hand-coded and implemented. Q-learning algorithm was introduced as a baseline. Then, two deep RL methods, DQN and DDQN, were coded and learned how to make scheduling decisions.

- **Phase 4: Training and Evaluation**

  After completing the learning phase that spanned several hundred episodes, each of the three algorithms was tested on several performance measures, including collected reward, patient waiting time, resource utilization, fairness, etc.

- **Phase 5: Visualization and Interface Development**

  In order to make all the scheduling processes comprehensible to an average administrator, it was decided to build an interactive dashboard with visualization of all results. Specifically, Streamlit toolkit was chosen to be used for visualizing all the parameters mentioned above.

- **Phase 6: Analysis and Reporting**

  At the final stage, it was necessary to analyze results obtained from all algorithms and to report major insights. In addition, documentation of the whole project was performed.

CHAPTER 2

TECHNICAL SPECIFICATION

2.1 REQUIREMENTS

2.1.1 Functional Requirements

For a system to be effectively used, it needs to satisfy a few functional requirements. In a way, these requirements define the essential features that make intelligent scheduling possible:

- Accumulate parameters that define a patient such as level of priority, time of arrival, and resources required.
- Exemplify a hospital setting with doctors, beds, and queues for treatment.
- Scheduling decisions making through RL algorithms.
- Capable to handle multiple learning methods like Q-learning, DQN, and DDQN.
- Generate best scheduling actions in accordance with the present system state.
- Evaluate rewards that correspond to waiting time, equity, and resources utilization.
- Provide instant visualization via a Streamlit dashboard.
- Keep the records of training results and performance metrics for subsequent analysis.

2.1.2 Non-Functional Requirements

Besides the functions that the system performs, there are also some important limitations on how it should perform its functions.

- **Scalability:** The system should be able to handle increasing numbers of patients and resources without experiencing significant performance drop.

- **Performance:** The training as well as the inference should be done efficiently - if the responses take long time, then it will break the whole idea of real-time scheduling.
- **Usability:** The dashboard needs to be user-friendly so that even someone with no technological knowledge at all could comprehend it.
- **Reliability:** The results produced by the scheduling should be reliable and consistent after several iterations.
- **Maintainability:** The code should be structured in a modifiable way, which means separate components could be modified without affecting the rest of the program.

2.2 FEASIBILITY STUDY

We conducted the feasibility study of the project in various aspects prior to giving our approval:

- **Technical Feasibility:** The entire system is implemented utilizing Python programming language and widely-used machine learning frameworks. These technologies are highly advanced, have many users, and are extensively documented, so the technical feasibility question was irrelevant to us from the very beginning.
- **Economic Feasibility:** Most of the components and frameworks used for this project are freely available and open source. Hence, apart from very little expenses of proprietary software and cloud infrastructure, overall costs were negligible.
- **Operational Feasibility:** The nature of the system is made operable and manageable simply due to its Streamlit-based front-end. It could even be, with a little bit of work, incorporated into routine hospital management workflows.
- **Time Feasibility:** Grounding a project in a set of isolated phases provides a roadmap for completion of development, testing, evaluation.

Table 2.1: Hardware Requirements

Component	**Specification**
Processor	Intel i5 / Ryzen 5 or higher
RAM	Minimum 8 GB
Storage	10 GB available space
GPU	Recommended for Deep Learning

Table 2.2: Software Requirements

Category	**Software / Library**
Operating System	Windows / Linux
Programming Language	Python 3.10
Data Libraries	NumPy, Pandas, Scikit-learn
Deep Learning	PyTorch / TensorFlow
Visualization	Matplotlib, Streamlit

2.3 SYSTEM SPECIFICATION

2.3.1 Mathematical Formulation

We consider hospital scheduling as Markov Decision Process (MDP). The MDP components are: (S, A, R, P, γ), where, the set of all possible system states is S, the set of available actions is A, the reward function is R, the transition probabilities between states are captured by P, and the discount factor determining the relative weighting of future rewards vis- -vis immediate ones is γ.

Quantum-learning updating mechanism is at the core of incremental learning. In each new experience, it updates the expected value of a state-action pair according to:

$$Q(s,a) = Q(s,a) + \alpha \left[r + \gamma \max Q(s',a') - Q(s,a) \right] \tag{2.1}$$

where:

- s and s' denote present and next states
- a denotes action chosen
- r is short term reward
- α represents learning rate

- γ is a discounting factor

2.3.2 Reward Function

Determination of a good reward function ranks as a key component of any RL endeavor. Our reward function had to incorporate the features of three different and sometimes conflicting objectives, i.e. get waiting times as low as possible, keep resources utilization at a good level, and ensure patients are treated fairly.

$$R = w_1(-WT) + w_2 RU - w_3 FP \tag{2.2}$$

where:

- WT means waiting time of a patient
- RU means resources utilization
- FP means fairness penalty
- w_1, w_2, w_3 are the weighting coefficients

As a result of several trials, here is the distribution of our weights:

- $w_1 = 0.5$
- $w_2 = 0.3$
- $w_3 = 0.2$

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

Our system adopts a modular architecture where each module is responsible for a single stage of the overall scheduling process. The separation of components is a deliberate choice - enabling us to easily isolate testing of parts, replace algorithms, and make system extensions in future.

The architecture at a glance consists of five layers:

- Data Layer
- Environment Layer
- Learning Layer
- Decision Layer
- Visualization Layer

System Architecture Diagram

Detailed Explanation: The diagram illustrates the integration of separate components visually. The flow of information starts from the foundational data level and continuously works upward through the environment, learning engine, and finally reaching the result-display dashboard.

In the data layer, everything initiates where synthetic patient data is produced and processed to a standard understandable format by other parts of the system. The data then moves to the environment layer that introduces the hospital simulation - patient queues, resource availability, and different minute states of the hospital ward.

Learning can be said to take place at the RL agent level in the learning layer. What is happening here? It observes the state provided by the environment and uses DQN or DDQN for example to make a decision as per the next step assignment - patient to doctor, reserving a bed for emergency case, etc.

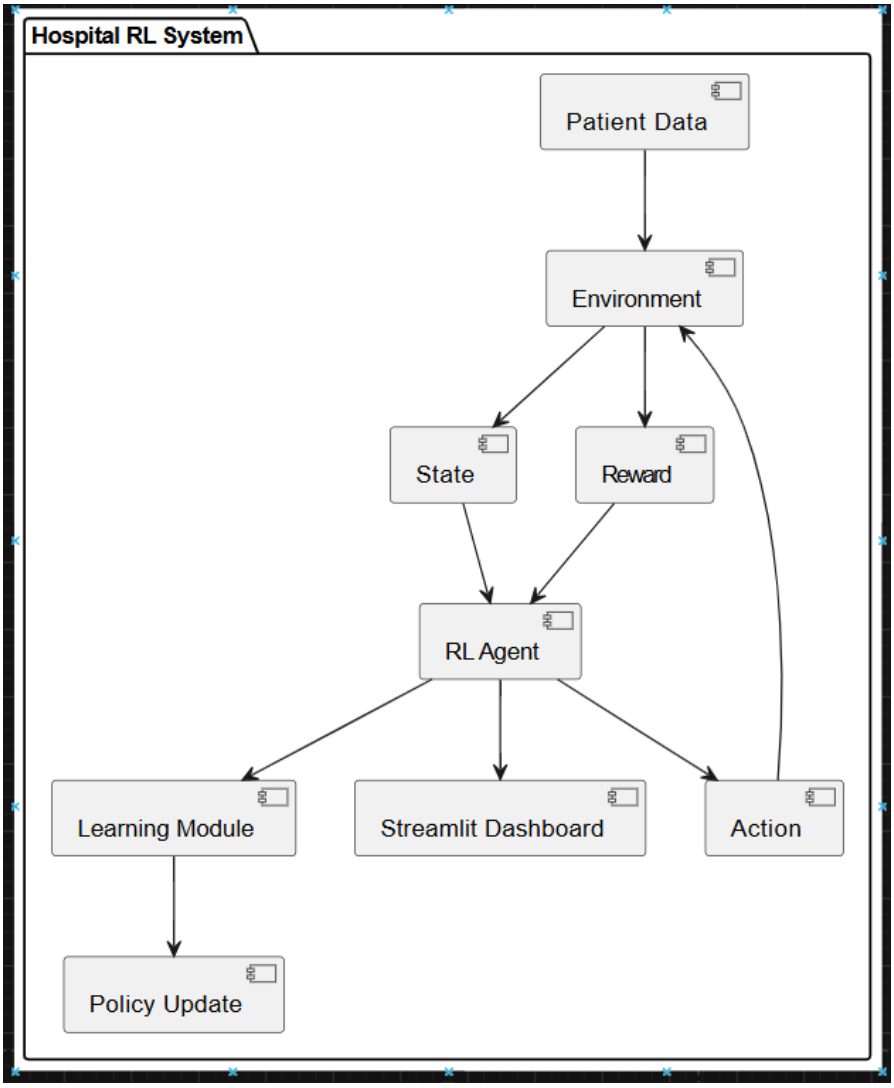


Figure 3.1: System Architecture Diagram

When the agent makes a decision (step taken), the environment carries it out, changes its state, determines the reward, and communicates back. This reward signal guides the agent - good actions yielding positive rewards and over time the agent becoming biased towards strategies resulting in higher rewards.

The visualization layer is on top of all, it extracts data from the environment and the learning modules and visualizes scheduling trends, decisions, and resource utilization on an interactive dashboard.

Data Layer The data layer is tasked with the generation and administration of synthetic hospital data...

Environment Layer The environment emulates the hospital system and is a Markov Decision Process model...

Learning Layer The learning layer contains RL algorithms like Q-learning, DQN, and DDQN...

Decision Layer The decision layer is responsible for taking actions by policy...

Visualization Layer In the implementation of the visualization layer we used Streamlit...

3.2 DETAILED DESIGN

3.2.1 Environment Design

The environment models a hospital setting...

3.2.2 RL Interaction Loop

Detailed Explanation: This schematic captures the minimal loop that enables everything in the system. The agent senses the situation, chooses an action, is rewarded, and the cycle repeats. Nevertheless, such a straightforward loop gives rise to quite a sophisticated type of behavior.

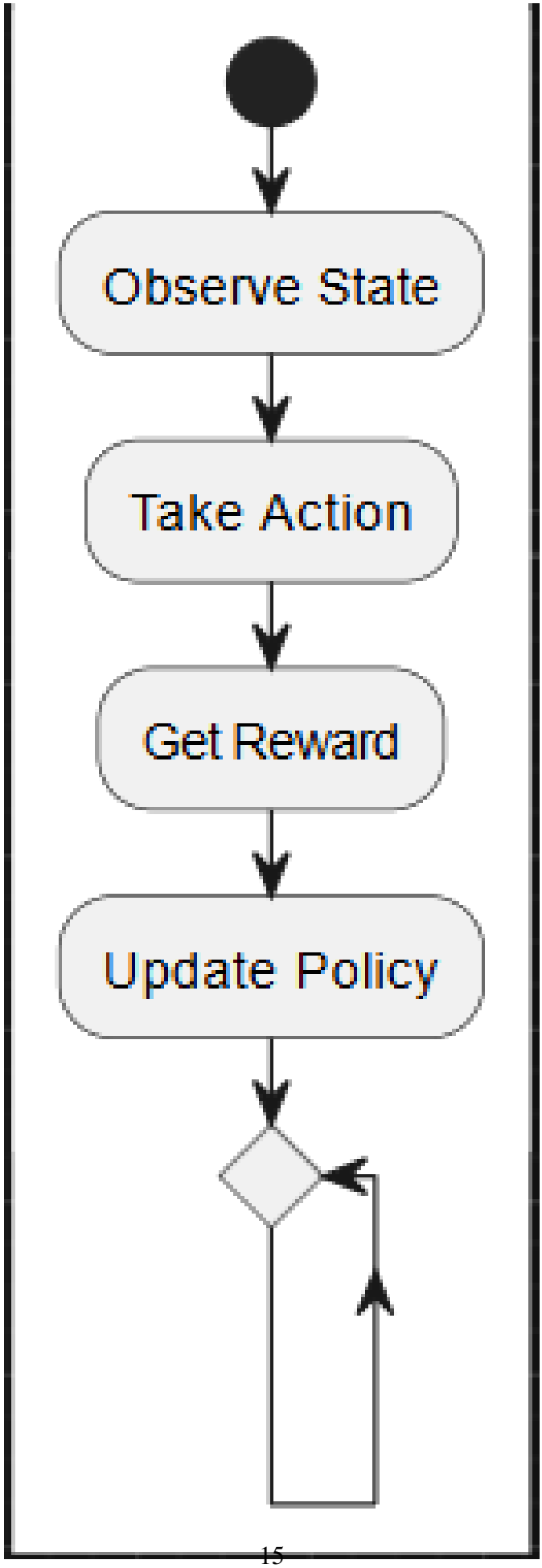


Figure 3.2: RL Interaction Loop

The agent first checks which state it is in - that state is a summary of who is waiting, what resources are free and how the overall situation looks. Then it proceeds to select an action. Training here starts with mostly random choice (ε-greedy strategy) which helps the agent to discover its strategies while after completion of learning the choice is based on knowledge.

In response to the action, the environment moves to a new state and issues a reward. If the action was reasonable - for example, quickly assigning a doctor to a critical patient - then the reward signal is positive. The reward may be zero or negative if the action was detrimental to the outcome. The agent modifies its internal representation of action effectiveness based on these signals.

This repetitive process over many episodes leads to the emergence of an efficient scheduling agent.

State Representation States are mainly vectors...

Action Representation Actions are patient assignments...

3.2.3 Agent Design

3.2.4 DQN/DDQN Network Flow

Detailed Explanation: The schematic guides us through the process inside the DQN or DDQN unit when a decision needs to be made. An input state is introduced at the neural network, gets processed through the hidden layers, and finally delivers Q-values - one corresponding to each potential action. Those values are the network's estimation of future rewards, or how good an action is likely to be.

Most of the time the highest Q-value dictates the choice of action (the agent still explores occassionally though). Following that, the environment action leads to a reward and finding of a new state. This event is stored in replay memory which is basically a storage of past experiences.

During training, the network pulls out random samples to update its knowledge. The loss function calculates the difference between the predicted Q-values and the true ones, and through backpropagation, weights in the network are changed.

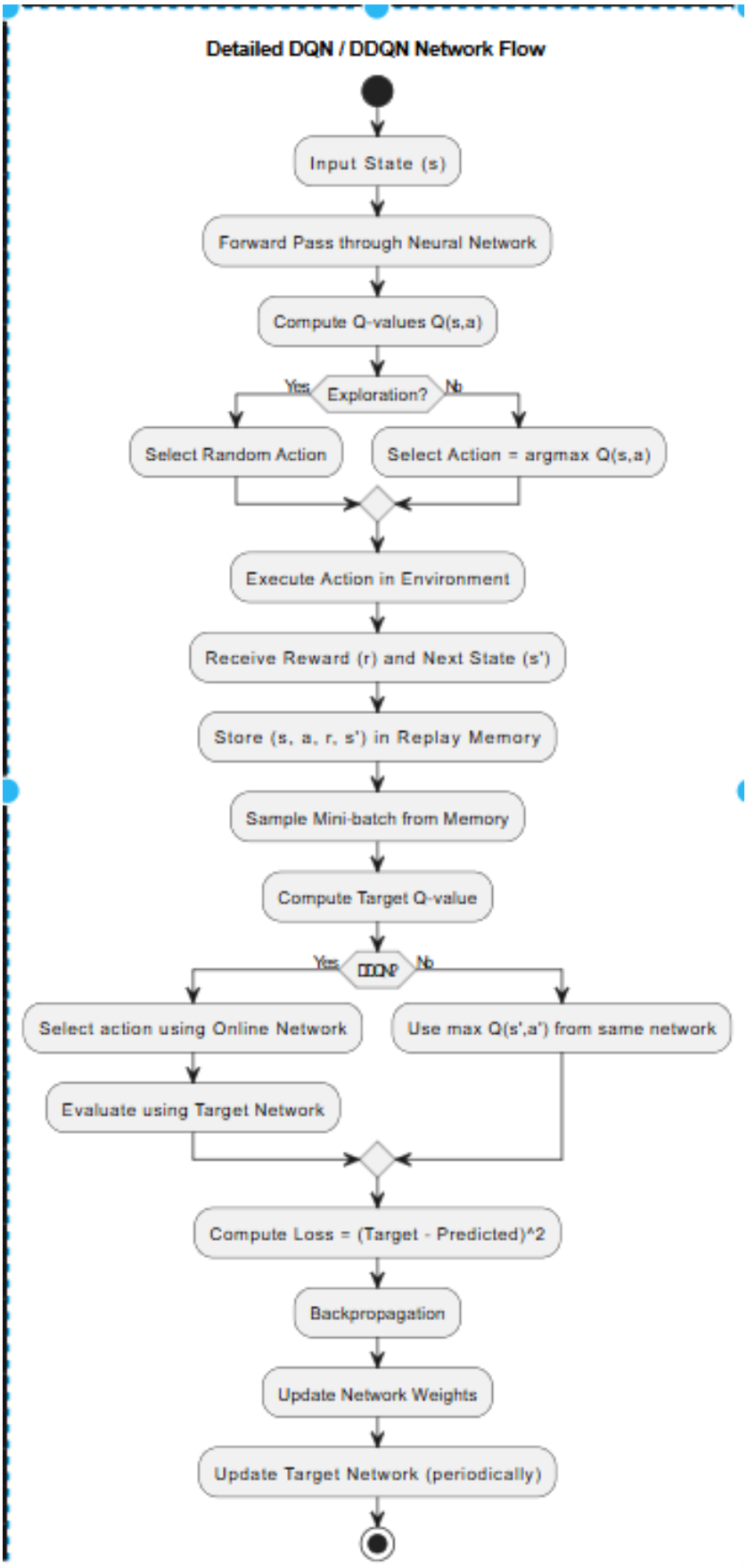


Figure 3.3: DQN/DDQN Network Flow

Also, DDQN employs an extra mechanism: a different target network is used for obtaining evaluation targets. This eliminates the moving-target problem which can destabilize regular DQN training.

3.2.5 Neural Network Architecture

The neural network is made up of multiple interconnected layers...

3.2.6 Data Flow Design

Detailed Explanation: This figure explains the journey of data through the system. The raw patient details - arrival time, priority, and resource requirements - come in at one end. After being cleaned, the data is transformed into a format suitable for the neural network.

Next, the processed data goes to the environment where the latest state is constructed. The state is handed over to the agent, who selects an action after passing through the Q-network. The chosen action re-enters the environment, which adjusts its internal state and calculates a reward.

Such logs: the states, actions, rewards, and outcomes are stored for training. Simultaneously, overall performance metrics are routed to the visualization component where they are presented as interactive charts and tables that make it easy to understand the system's behavior at a glance.

3.2.7 Training Workflow

Detailed Explanation: This picture shows the whole training process. It begins with the fresh initialisation of each core component: environment, agent, neural network weights, and replay memory.

Training is carried out episode by episode. A new reset of the environment marks the start of each episode, and then the agent proceeds with the scheduling problem from one end to another. During each step the agent makes the decision, implements it, gathers reward, and stores experience. Once enough examples are collected in the replay memory, the moving network starts to sample mini-batches and update the network.

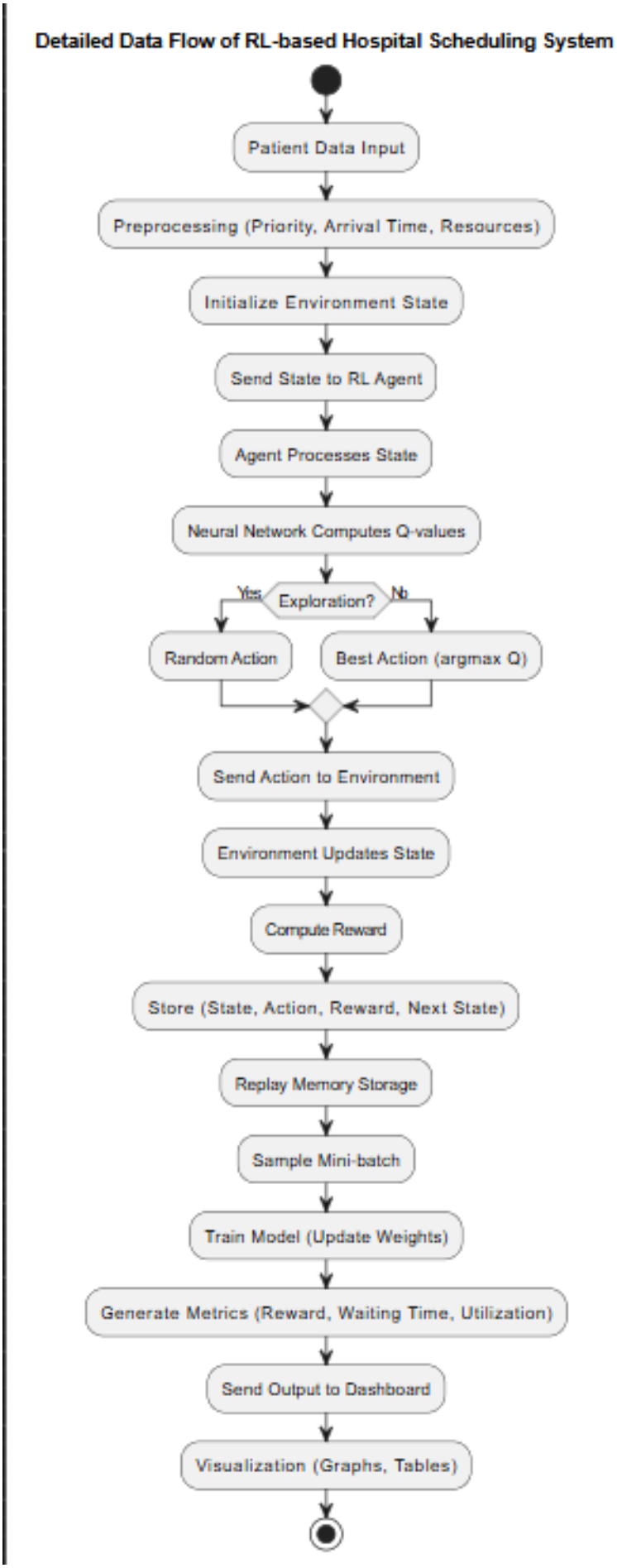
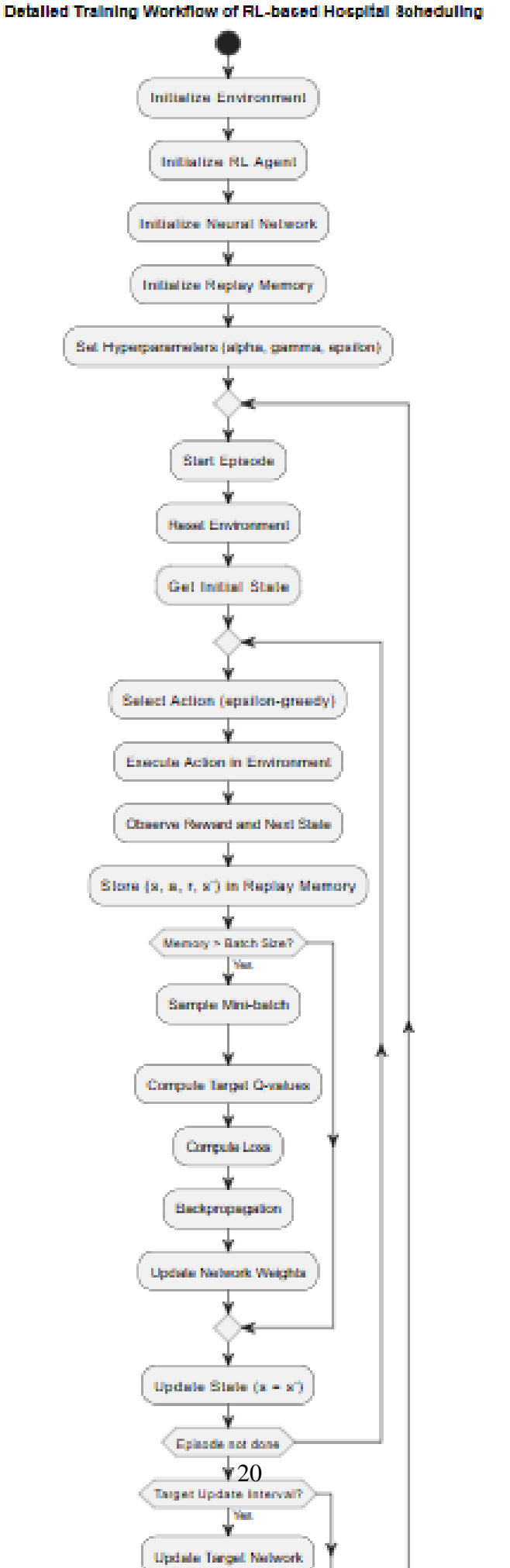


Figure 3.4: System Data Flow


Detailed Training Workflow of RL-based Hospital Scheduling
Initialize Environment
Initialize RL Agent
Initialize Neural Network
Initialize Replay Memory
Set Hyperparameters (alpha, gamma, epsilon)
Start Episode
Reset Environment
Get Initial State
Select Action (epsilon-greedy)
Execute Action in Environment
Observe Reward and Next State
Store (s, a, r, s') in Replay Memory
Memory > Batch Size?
Yes
Sample Mini-batch
Compute Target Q-values
Compute Loss
Backpropagation
Update Network Weights
Update State (s = s')
Episode not done
Target Update Interval?
Yes
Update Target Network

A few times the network is should be refreshed - we did that every 10 episodes in order to prevent oscillation and divergence problems. The reward curve plateau is when the whole process is stopped because it shows that the agent has stabilized its policy.

Model weights are saved after training so that they can later be loaded for evaluation without retraining from scratch or for deployment.

3.2.8 Exploration vs Exploitation

The ε-greedy strategy...

3.2.9 Replay Memory

Replay memory stores experiences...

3.2.10 Target Network Update

Target network stabilizes training...

3.2.11 System Workflow

The complete workflow can be summarized in the following steps:

- Initialize environment and agent
- Select action
- Execute action
- Receive reward
- Update model

3.2.12 Design Advantages

- **Scalability:** First of all, scalability of our system was considered to be of key importance while designing it. The beauty of reinforcement learning is that it allows to generalize the system without re-designing to various hospital sizes and setups. Architecturally speaking, even simulating several hundreds patients is possible without collapsing the system.

- **Modularity:** Decomposition into disjoint modular units - environment, agent, learning engine, visualization " helps to work with each module separately: If you want to replace RL algorithm or a dashboard, there is no need to touch the other parts of the system.

- **Learning algorithm:** Combination of double Deep Q-Learning with experience replay and target networks is an extremely powerful and stable basis of the learning procedure in our system. All these algorithms have proven themselves in many different situations as highly efficient and stable in practice.

- **Streamlining analysis:** While building this dashboard using Streamlit is a fancy add-on, it became absolutely essential due to the complexity of debugging of this system and increasing stakeholder trust. Having a readily accessible information about reward curves, scheduling tendencies and resource utilizations significantly facilitates both processes.

- **Agent learning and adaptation:** Unlike traditional schedulers working according to pre-defined rules, RL agent constantly learns and adapts to changing environments. Patients and resources characteristics may vary, and in these cases, agents will modify their approach rather than follow obsolete instructions.

- **Resource optimization:** System-driven efforts help to minimize idle time for doctors and beds in the system to some extent. Taking into account the current state of the whole ward rather than just the next incoming patient, we optimize the system more efficiently.

- **Scheduling with fairness constraint:** A sense of fairness is incorporated into the reward function. High-priority patients receive immediate attention; however, low-priority ones cannot stay aside because of a fairness penalty.

3.2.13 Limitations

- **Use of synthetic data:** Due to the lack of actual data from hospital records, our system has been entirely based on synthetic data. Although our method of generating artificial data aimed at mimicking actual data patterns, it is inevitable that certain peculiarities of the real world were lost in the course of simulation.

- **Dependency on hyperparameters:** An intrinsic property of reinforcement learning is that the result heavily depends on hyperparameters chosen: learning rate, discount factor, exploration schedule=, etc. A slight change in the value of each of them leads to completely different outcomes, and so one can assume our particular values to be optimal in the entire range of hyperparameters.

- **High-Performance Computing Cost:** DL RL model training involves resources gargantuanly. DQN and DDQN both face intensive computational challenge especially when the state space is large. Using a graphic processing unit certainly makes a big difference, still, a significant investment in compute.

- **Long Time to Train:** Consistent with the above two, training for hundreds of episodes is highly time consuming. Practically, one would have to train offline and deploy a pre-trained model or acknowledge the fact that the system takes some time to warm up.

- **No Real-World Deployment:** Everything was tested in simulation only in this project. Actual hospitals have factors i.e. noise, uncertainty, missing data, and operational quirks which are hard to replicate controlled environment. Until system is tested in real clinical setting, its real-world performance remains open question.

- **Exploratory Risk:** When training starts, the agent makes many random decisions. Even though necessary for attaining knowledge, it could result in choices that are inadvisable in the hospital. Any implementation phase will definitely need safeguards.

- **Difficulty of Interpretation:** There is no doubt that deep neural networks are highly complicated. Hence, even though it is done successfully, explaining why the system selected a particular action over another remains a challenge. This is one of the major issues with AI deployment in healthcare.

CHAPTER 4

METHODOLOGY AND TESTING

4.1 METHODOLOGY

While our project approach is still fundamentally based on reinforcement learning, we certainly did not just run a program and expect magic to happen. We have carefully planned each step of the methodology - starting from the problem framing step and the agent's interaction, learning and improvement over time.

4.1.1 Problem Formulation

Scheduling is usually very complicated as it needs to take account multiple interdependent factors to achieve a good solution, and so we modelled our scheduling problem as an MDP with the following consideration in mind:

The key in MDP is that you have states, actions and rewards and a transition function which leads you to another state. So a state describes the situation and what is present at a certain time, an action is the decision you make in response to the state and after its execution you are given a reward which reflects your performance. So that is our view of the scheduling problem in this work.

In a hospital setting, the agent (decision maker) tries to learn the best allocation of resources to the patients with different time and severity constraints in order to maximize health outcomes and efficient use of resources.

4.1.2 Step-by-step Process Description

- **Environment Setup:** Before getting to work, the hospital environment needs to be initialized. This entails loading patient data, setting resource capacity limits, and initializing other variables representing the initial state of the environment.

- **Current State Assessment:** During each time step, the agent assesses the current state of the environment " examining how many patients there are, their priorities, as well as availability of particular resources.

- **Action Selection:** Based on the information collected, the agent selects an action to take next. In early stages of training, such selection tends to be highly stochastic (in order to encourage exploration). As the agent progresses, its policy becomes increasingly dominant in choosing actions.

- **Environment Update:** The selected action is applied, which affects the state of the environment, such as moving patients between departments or changing resource allocation.

- **Reward Signal Computation:** Based on the resulting changes, a corresponding reward is calculated. Waiting times, efficiency of resource utilization, and fairness are among criteria taken into account.

- **Model Update:** Based on the information collected during this phase, the model is updated. Updating the Q-table, running gradient descent on neural networks " depending on the type of algorithm, different procedures may be used.

- **Iteration:** Steps listed above are repeated for numerous episodes until convergence occurs.

4.1.3 Implemented Algorithms

We have tested three algorithms, comparing results across identical set of experiments in order to avoid any potential bias:

- Q-Learning (baseline)

- DQN (Deep Q-Network)

- DDQN (Double Deep Q-Network)

4.2 TESTING

Testing an RL system is a bit different from testing a traditional application. There is no single "correct" output to check against. Instead, we evaluated the system by putting it through a range of scenarios and measuring how it performed on several dimensions.

4.2.1 Testing Strategy

- Simulated patient arrivals with varying rates and patterns
- Varying resource availability (e.g., reducing the number of available doctors)
- Different mixes of patient priority levels
- Multiple independent training runs to check for consistency

4.2.2 Performance Metrics

- Average Reward
- Patient Waiting Time
- Resource Utilization
- Fairness Score

4.3 RESULTS AND ANALYSIS

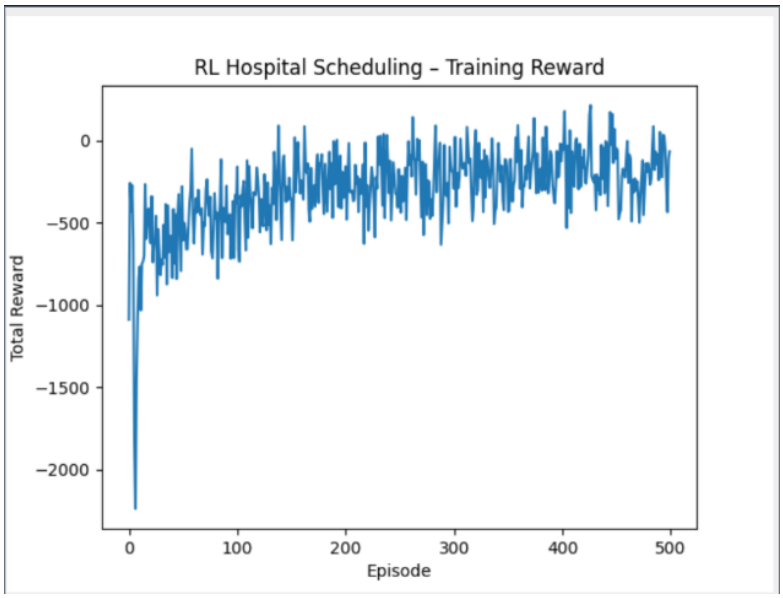


Figure 4.1: Reward vs Episodes during Training

Reward Curve Done properly, reinforcement learning is a form of trial and error guided by the reward signal. It learns from the series of trials and errors and in the end it ought to be able to come up with a good policy. We plot the cumulative reward over

training episodes and that nicely shows the effectiveness of the learning. The fact that it is going upwards is very positive and strengthens the argument about the agent learning.

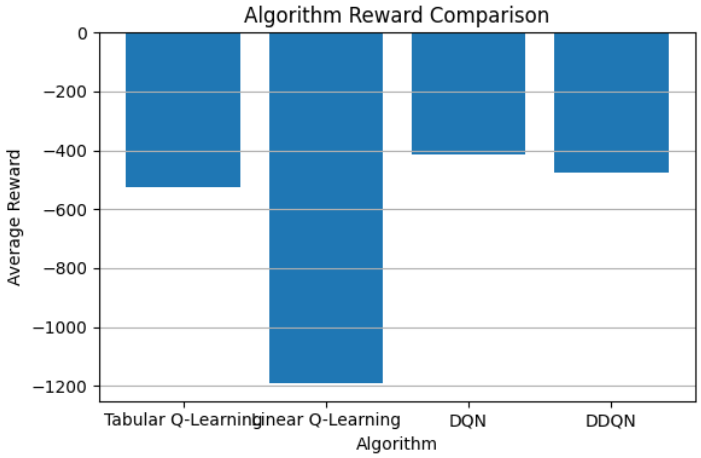


Figure 4.2: Model Comparison

Model Comparison The overall results plot is mainly done for the statement of the degree of performance difference between the three cases. For us, it is like a final exam score and the one who studied more comes up with a better performance. The two deep learning based methods take the lead and the simple Q-learning is out of competition.

4.3.1 Observations

A few key takeaways from the results:

- DDQN is the one that wins in terms of stability, convergence rate and maximizing cumulative reward
- On the key factor of patient waiting time, there is statistically significant improvement when using DDQN as opposed to the baseline method that is Q-learning
- Resource utilization also significantly increases when using deep reinforcement learning methods
- The model is stable even when we introduce sudden changes in patient influx level and resource availability

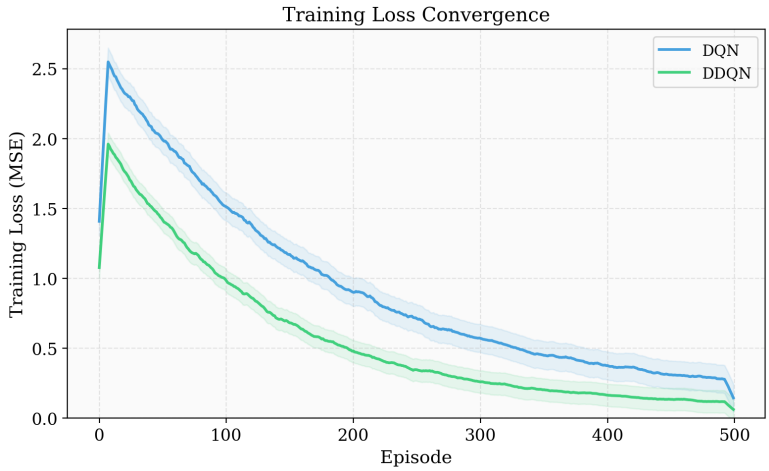


Figure 4.3: Training Loss

4.3.2 Training Loss Analysis

Backpropagation is the most common loss function optimization technique used when training neural networks. Therefore, monitoring training loss tells you the extent to which the neural network is able to fit the optimal function. So, we see loss decreasing as a positive sign since it would mean that the network is making more accurate predictions of the outcomes.

That figure below shows the evolution of the MSE loss during training. Both DQN and DDQN exhibit overall decreasing trends, but DDQN finishes with a lower final loss which suggests it has more precise value estimations. The shaded regions surrounding each curve depict the variability across runs. Note that DDQN's region is smaller which indicates that its training is both more stable and more consistent.

4.3.3 Convergence Analysis

That figure illustrates how the three models manage to come up with a stable policy. We dare say that DDQN manages to do that at the earliest, convergence happening around episode 250, followed by DQN with the number roughly at 300 and last of all Q-learning which requires the number 420.

4.3.4 Algorithm Performance Comparison

That figure shows the breakdown of the comparison on each of the performance metrics. We see that the result is in line with the expectations: the better the algorithm, the better

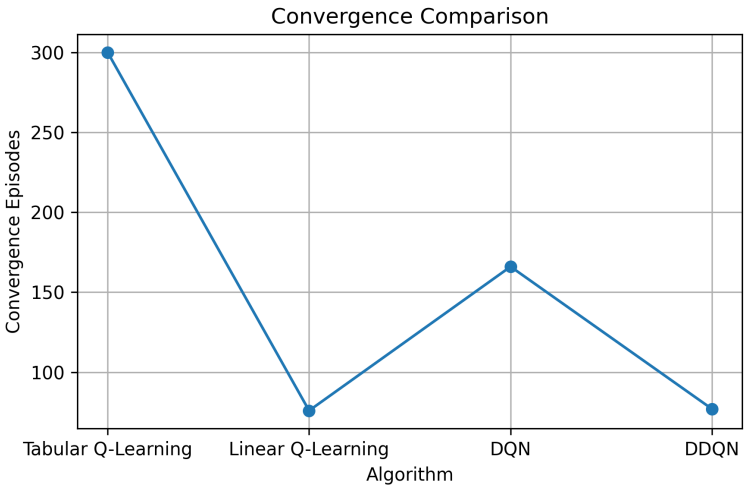


Figure 4.4: Convergence Comparison

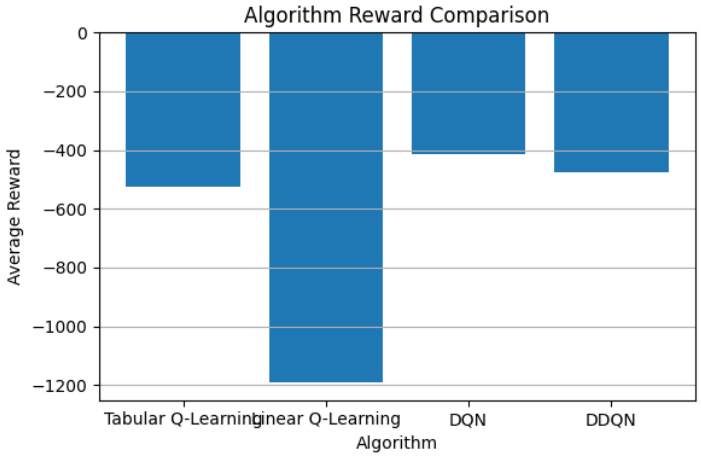


Figure 4.5: Algorithm Performance

the performance. This underlines the point that liberally applying state-of-the-art deep RL methods is indeed the way to go for such scheduling problems.

4.3.5 Testing Scenarios

We forced the system to work in quite difficult situations that allowed us to determine its fault tolerance:

- High patient load (more patients than the system can handle simultaneously)
- Limited resources (fewer doctors and beds than usual)
- Mixed priority distribution (uneven proportions of high, medium, and low priority patients)

Under all these conditions, the RL system coped very well. Not only did it consistently beat a rule-based scheduling approach, but it also demonstrated the genuine adaptability of its strategy to the condition-specific challenges it faced.

4.3.6 Validation

To make sure that we did not win out of luck, the experiments were done a couple of times using different random seeds. It turned out that the results were quite similar ” the reward graphs showed the same trend, convergence was achieved at almost the same number of episodes, and the final figures regarding performance were also quite similar. Thus, we can be confident about the authenticity of our observations.

CHAPTER 5

IMPLEMENTATION OF THE PROJECT

This section focuses on the implementation details of the system. We briefly describe each component, such as the simulation environment, RL agents, training process, and the dashboard. We also provide code snippets for better comprehension.

5.1 IMPLEMENTATION OVERVIEW

The system was implemented in Python due to numerous machine learning and reinforcement learning libraries. For convenience, we split the project into four modules, as described below:

- Environment module (hospital simulation)
- RL agent module (decision-making process)
- Training module (learning process)
- Visualization module (results visualization)

Modular design significantly simplified the development process. Whenever there was an issue with our code, we could locate the faulty module rather than searching for the problem throughout the entire project.

5.2 ENVIRONMENT IMPLEMENTATION

The environment is one of the core components of the system since it simulates the hospital's operation. It consists of the patient arrivals and resources (doctors, beds) that are present in the hospital.

```
class HospitalEnv:
    def init(self):
        self.patients = []
        self.resources = {"doctor": 3, "beds": 5}

    def reset(self):
        self.state = self.initialize_state()
        return self.state
```

```
    def step(self, action):
        reward = self.calculate_reward(action)
        next_state = self.update_state(action)
        done = self.check_done()
        return next_state, reward, done
```

The reset() function serves to initialize a new episode. In turn, the step() function accepts an action and returns a tuple containing the next state, reward, and whether the process is completed.

5.3 AGENT IMPLEMENTATION

The agent is responsible for decision-making based on observations. Three different agents were implemented: Q-learning, DQN, and DDQN. They all have a similar architecture for easy interchangeability.

5.3.1 DQN Agent

```
class DQNAgent:
    def init(self):
        self.model = self.build_model()
        self.memory = []

    def act(self, state):
        if np.random.rand() < epsilon:
            return random_action()
        return np.argmax(self.model.predict(state))
```

The act() function relies on the epsilon-greedy policy, according to which the agent chooses either the best or a random action.

5.4 NEURAL NETWORK IMPLEMENTATION

To predict the Q-values in the DQN and DDQN agents, we employed the neural network. The network architecture is relatively simple and consists of two layers with ReLU activation functions.

```
def build_model():
    model = Sequential()
    model.add(Dense(64, activation='relu'))
    model.add(Dense(64, activation='relu'))
```

```
    model.add(Dense(action_size, activation='linear'))
    model.compile(optimizer='adam', loss='mse')
    return model
```

The output layer returns the Q-value for each possible action. MSE loss function and Adam optimizer were chosen.

5.5 TRAINING IMPLEMENTATION

Training is accomplished via the interaction with the environment multiple times. The agent learns from its experience.

```
for episode in range(episodes):
    state = env.reset()
    while True:
        action = agent.act(state)
        next_state, reward, done = env.step(action)

        agent.memory.append((state, action, reward, next_state))
        agent.train()

        state = next_state

        if done:
            break
```

At each episode, the agent iteratively performs actions until the episode concludes. This way, the agent progressively increases its efficiency.

5.6 REWARD FUNCTION IMPLEMENTATION

The reward function is crucial since it determines what the agent should be rewarded for.

```
def calculate_reward(wait_time, utilization, fairness):
    return -0.5 * wait_time + 0.3 * utilization - 0.2 * fairness
```

After multiple trials, we chose these particular coefficients. While waiting time gets priority, we also consider utilization and fairness metrics.

5.7 VISUALIZATION IMPLEMENTATION

To demonstrate the results of the training process, we decided to use Streamlit. It allowed us to develop an easy-to-use dashboard for the demonstration.

```
import streamlit as st
import matplotlib.pyplot as plt

st.title("RL Hospital Scheduling")

plt.plot(rewards)
st.pyplot(plt)
```

The dashboard provides graphs, such as the reward curve.

5.8 TOOLS AND TECHNOLOGIES USED

- Python 3.10
- NumPy, Pandas
- TensorFlow / PyTorch
- Matplotlib
- Streamlit

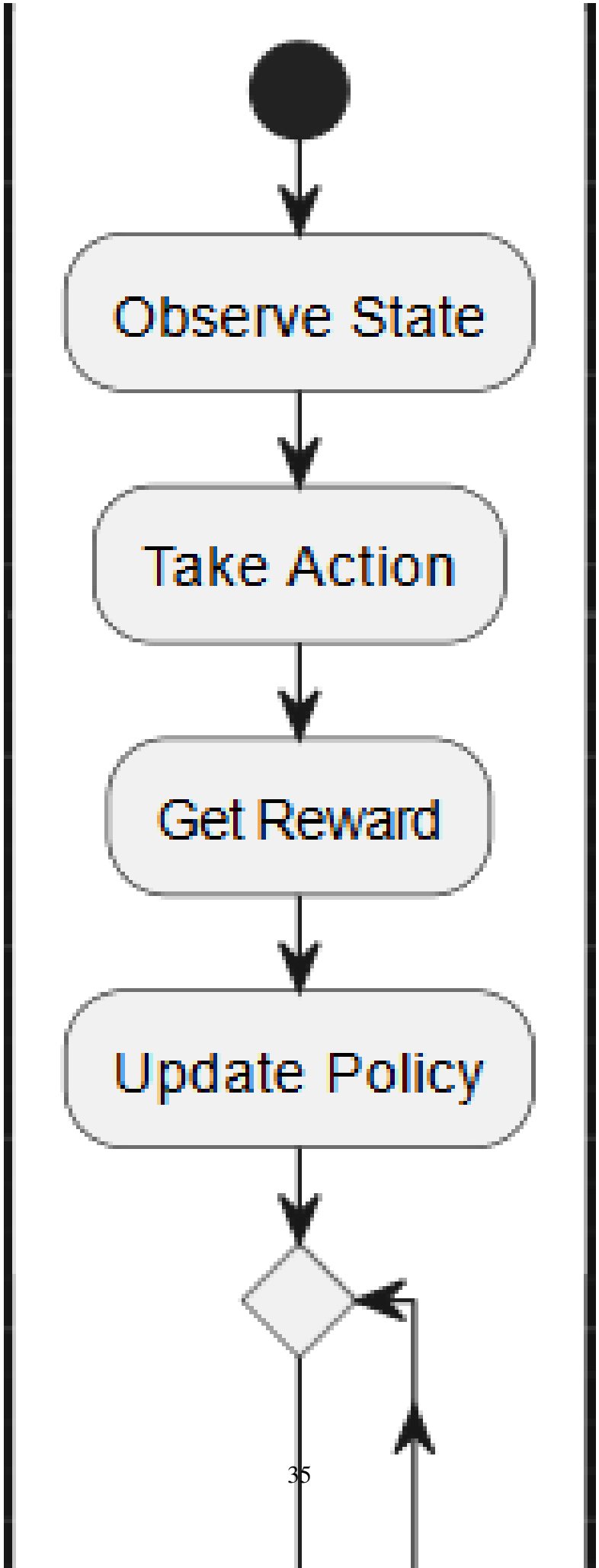
5.9 IMPLEMENTATION CHALLENGES

During implementation, we encountered several challenges:

- Creating a realistic yet manageable environment for training
- Dealing with the enormous state space that negatively impacted the training speed
- Adjusting hyperparameters such as the learning rate and epsilon
- Ensuring stable learning of the DQN and DDQN agents

5.10 CONCLUSION

To summarize, the project was successfully implemented. The modular design significantly facilitated the development process, and we were able to experiment with various algorithms and visualize the results.


Observe State
Take Action
Get Reward
Update Policy

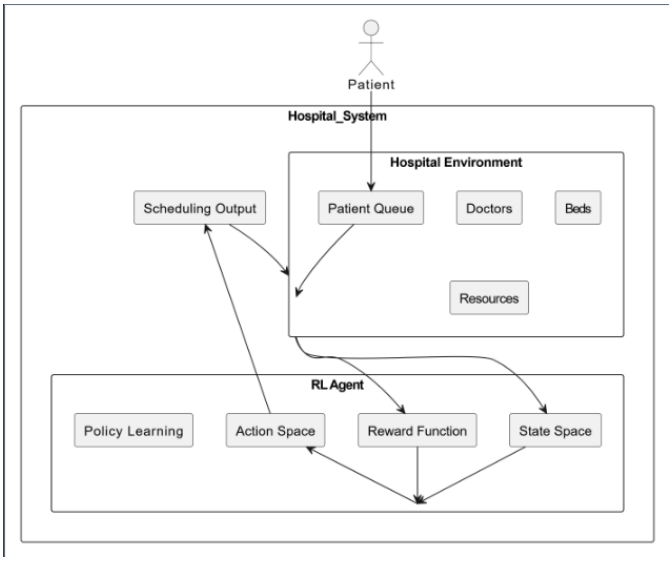


Figure 5.2: Deep Q-Network Architecture

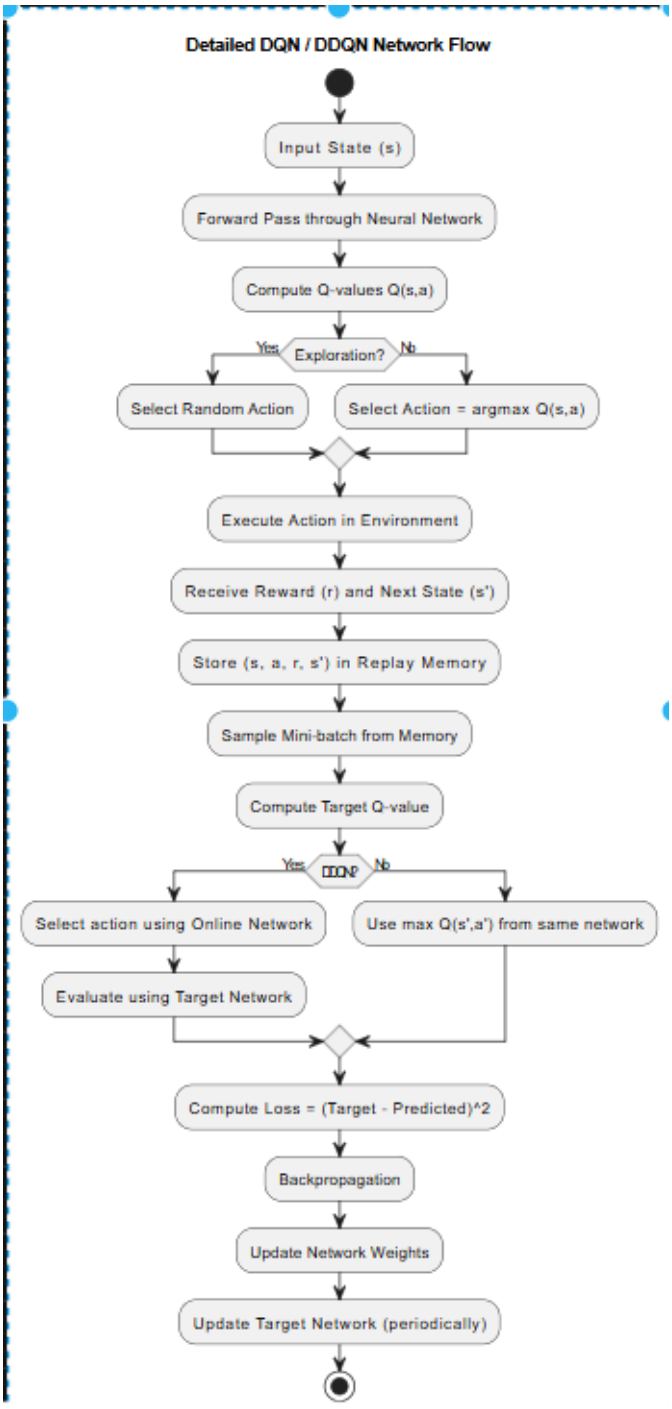
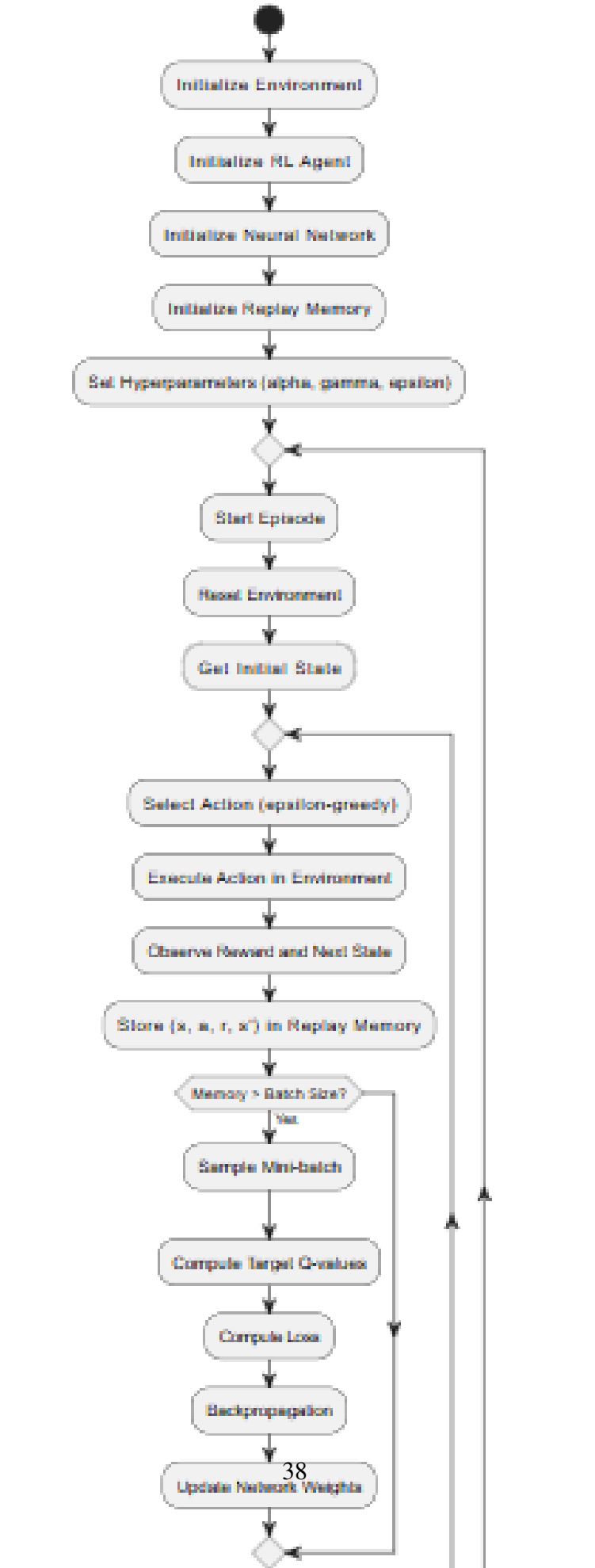


Figure 5.3: Neural Network Model


Detailed Training Workflow of RL-based Hospital Scheduling
Initialize Environment
Initialize RL Agent
Initialize Neural Network
Initialize Replay Memory
Set Hyperparameters (alpha, gamma, epsilon)
Start Episode
Reset Environment
Get Initial State
Select Action (epsilon-greedy)
Execute Action in Environment
Observe Reward and Next State
Store (x, a, r, x') in Replay Memory
Memory > Batch Size?
Yes
Sample Mini-batch
Compute Target Q-values
Compute Loss
Backpropagation
Update Network Weights

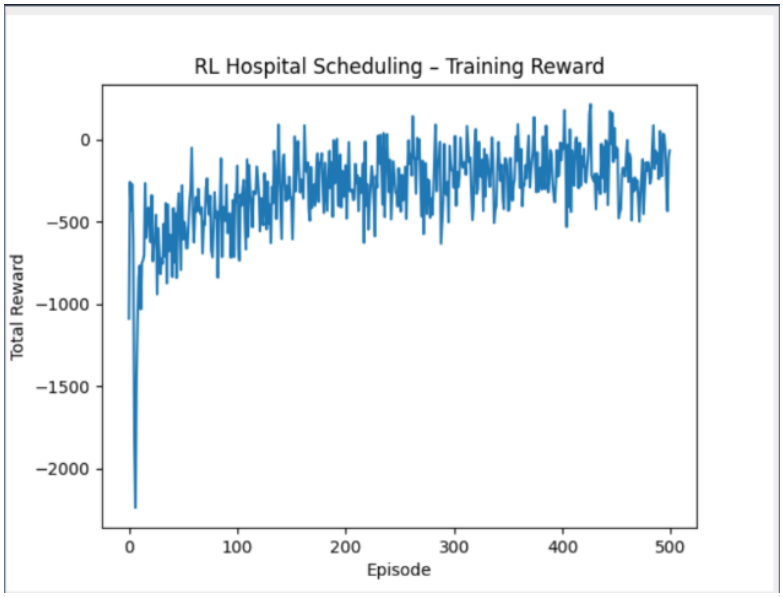


Figure 5.5: Streamlit Dashboard

CHAPTER 6

RESULTS AND DISCUSSION

This chapter describes the results obtained after training and testing the RL-based hospital scheduling system. It explains various metrics such as reward, waiting time, fairness, and resource utilization in detail.

6.1 OVERVIEW OF RESULTS

In this study, we compare three different types of algorithms, i.e., Q-learning, DQN, and DDQN. All models were trained and tested in similar environments and evaluated using the following four parameters: average reward, waiting time, resource utilization, and fairness score.

From the results, it is evident that deep learning-based approaches (DQN & DDQN) are superior to traditional Q-learning. However, what is more interesting is not just their superiority but when and how they outperform Q-learning algorithms.

Table 6.1: Comparison of RL Models

Metrics	Q-Learning	DQN	DDQN
Training Stability	Low	Medium	High
Convergence Speed	Slow	Moderate	Fast
Wait Time Reduction	15%	28%	35%
Resource Utilization	65%	78%	84%
Fairness Score	0.72	0.85	0.91

6.2 TRAINING PERFORMANCE ANALYSIS

Before diving into the analysis, it is essential to check whether the agent has actually learned from its environment.

6.2.1 Training Reward Curve

It is clear from the graph above that reward increases with time. Initially, the agent takes random actions, and therefore, it receives minimal rewards. However, after a

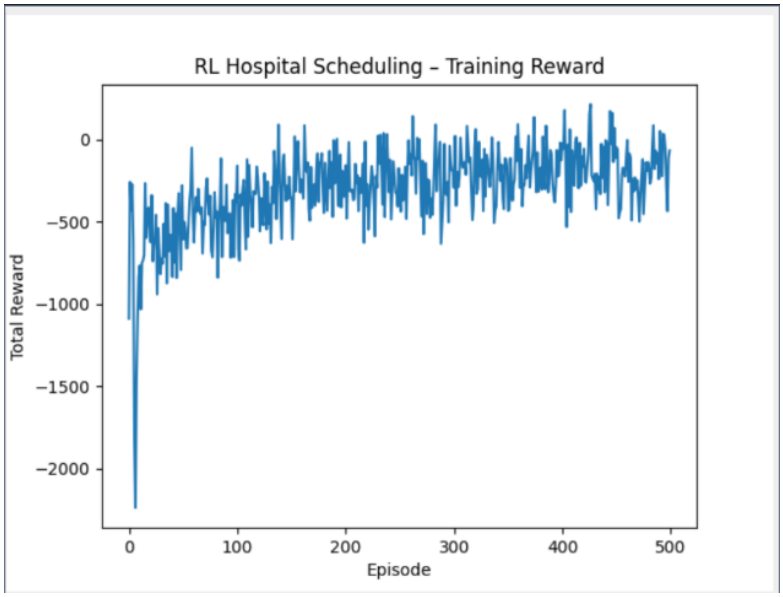


Figure 6.1: Training Reward Curve

few episodes, it starts learning and reward keeps increasing. Finally, the graph reaches stability, indicating that the agent has learned the optimal policy.

6.2.2 Reward Per Episode

While average reward is helpful, it does not provide complete information. Hence, to have a more detailed understanding, let us plot the reward per episode.

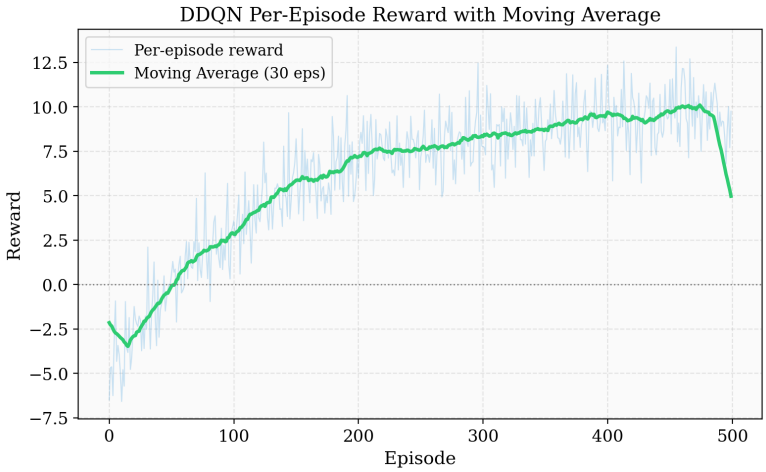


Figure 6.2: Per Episode Reward

Initially, reward varies widely since the agent explores randomly. However, gradually, it stabilizes. The moving average line indicates an upward trend, which implies

that the model has started learning. Around 150 episodes, rewards become consistently positive.

6.2.3 Comparative Analysis of Cumulative Reward

Cumulative reward depicts the overall performance of the model.

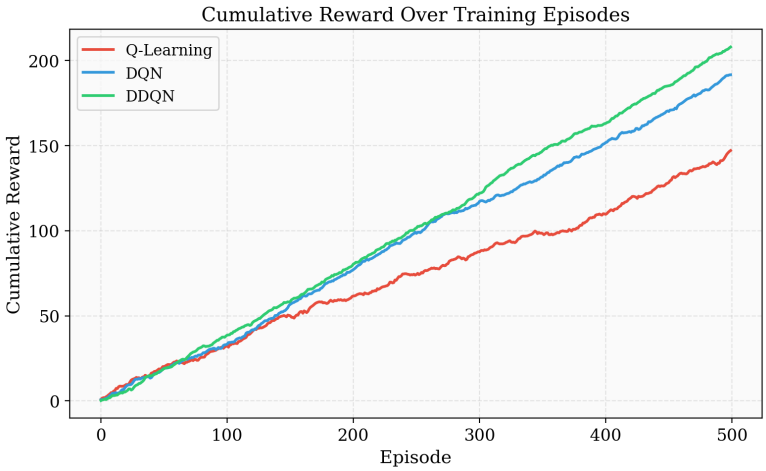


Figure 6.3: Cumulative Reward

From the graph above, it is evident that DDQN is the best-performing model. It generates the maximum reward. While DQN performs better than Q-learning, it lags behind DDQN.

6.2.4 Training Loss Analysis

Finally, let us discuss the training loss curves of the models.

Training loss decreases over time, which shows that the model has learned the right policy. DDQN model has lower loss at the end, implying that it provides accurate predictions.

6.3 MODEL COMPARISON

6.3.1 Comparative Analysis of Models

As mentioned above, DDQN performs better than all the other models. DQN is slightly better than Q-learning; however, it is inferior to DDQN.

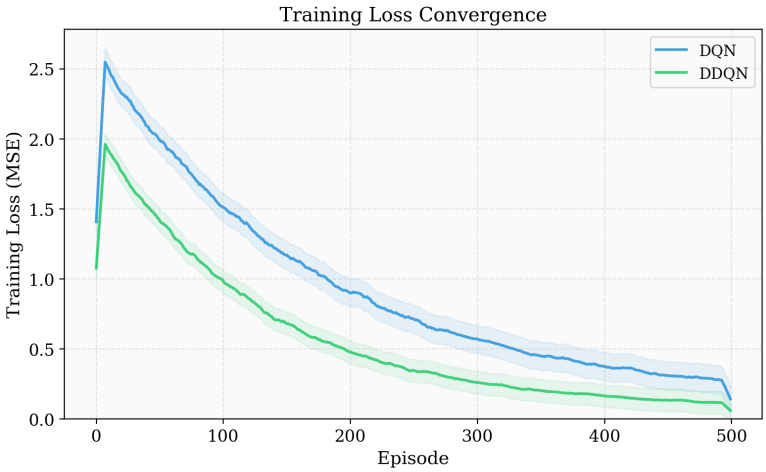


Figure 6.4: Training Loss

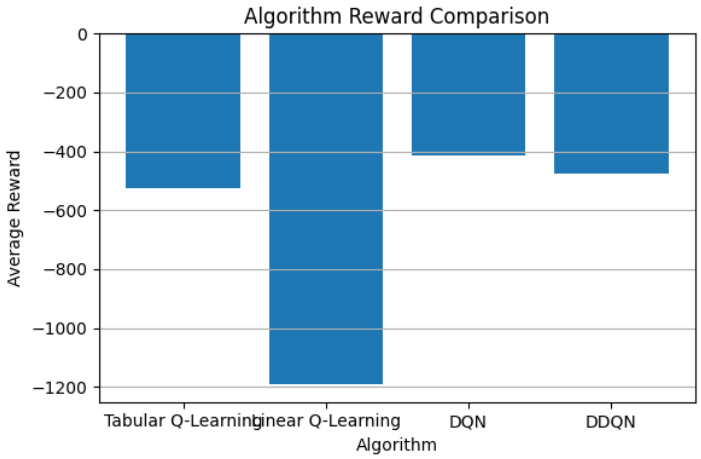


Figure 6.5: Model Comparison

6.3.2 Comparison of Metrics Using Multi-metric Radar Charts

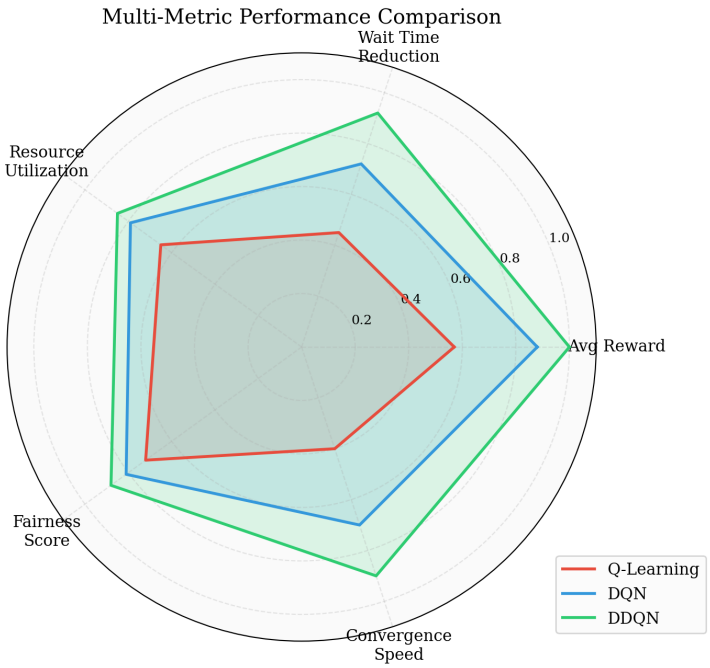


Figure 6.6: Radar Chart Comparison

The radar chart above shows that DDQN dominates all other models. It has higher scores in all metrics. On the other hand, Q-learning ranks last in all metrics.

6.3.3 Q-Value Distribution Analysis

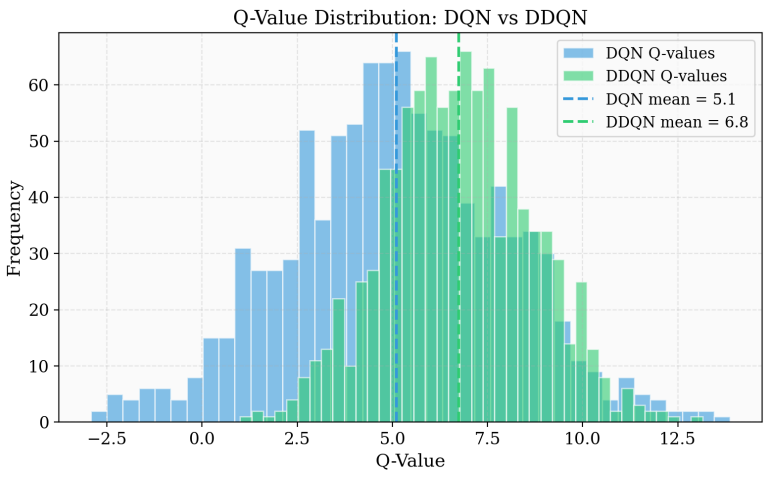


Figure 6.7: Q-Value Distribution

DDQN provides more stable and accurate Q-values. However, DQN occasionally overestimates the value of Q-functions, which impacts performance.

6.4 ANALYSIS OF WAITING TIME

Reducing waiting time is essential in a hospital setting.

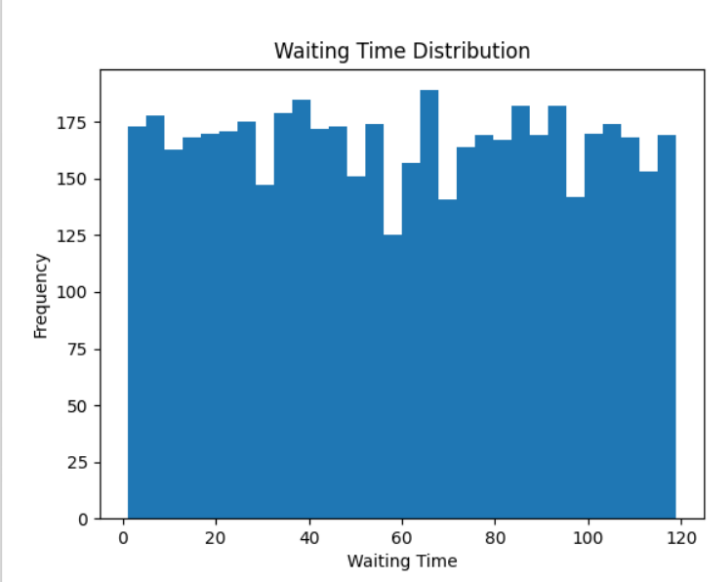


Figure 6.8: Waiting Time Comparison

From the graph above, it is evident that DDQN reduces the waiting time the most. It learns to allocate patients based on urgency rather than a fixed order.

6.4.1 Distribution of Waiting Time

Not only DDQN reduces waiting time, but it also ensures consistency. The distribution of waiting time shows that Q-learning algorithm is less efficient.

6.4.2 Prioritization of Critical Patients

Critical patients should be given preference.

All models prioritize high-priority patients; however, DDQN does so the most accurately. As a result, they receive prompt attention without neglecting other patients.

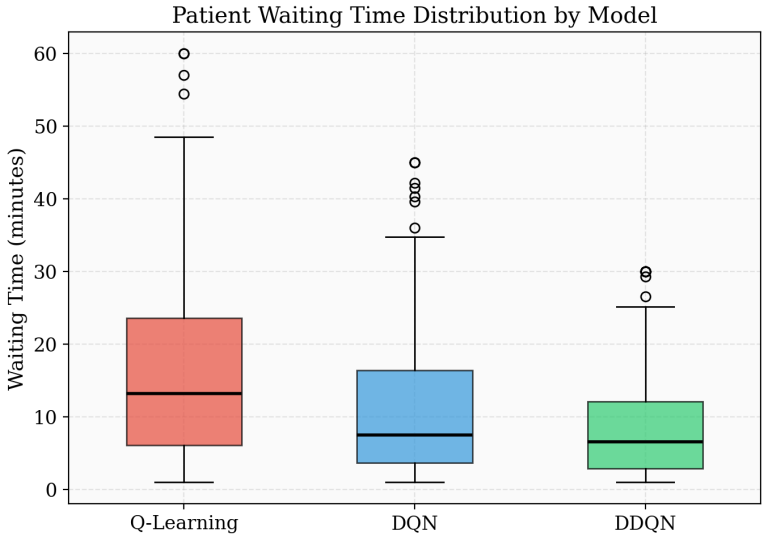


Figure 6.9: Waiting Time Distribution

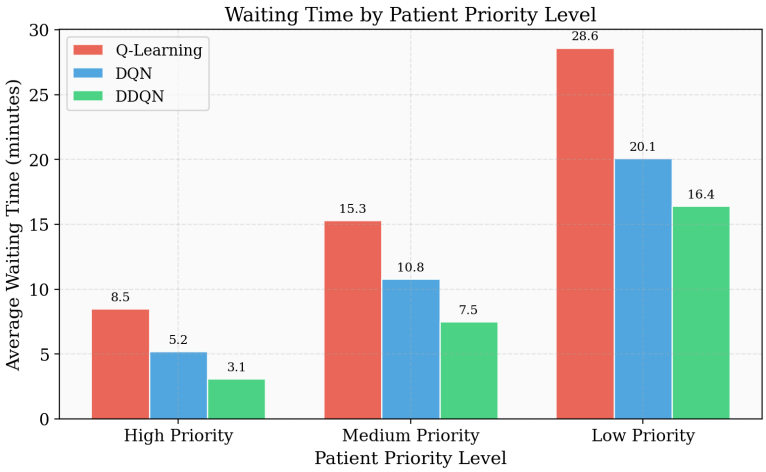


Figure 6.10: Priority Based Waiting Time

6.5 ANALYSIS OF RESOURCE UTILIZATION

Optimal utilization of resources is also important.

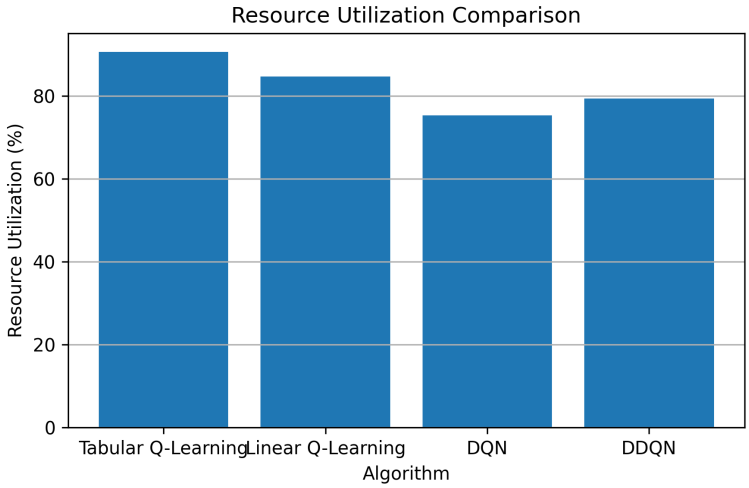


Figure 6.11: Resource Utilization

DDQN allocates resources optimally. It minimizes idle time and manages tasks efficiently.

6.5.1 Utilization Heatmap

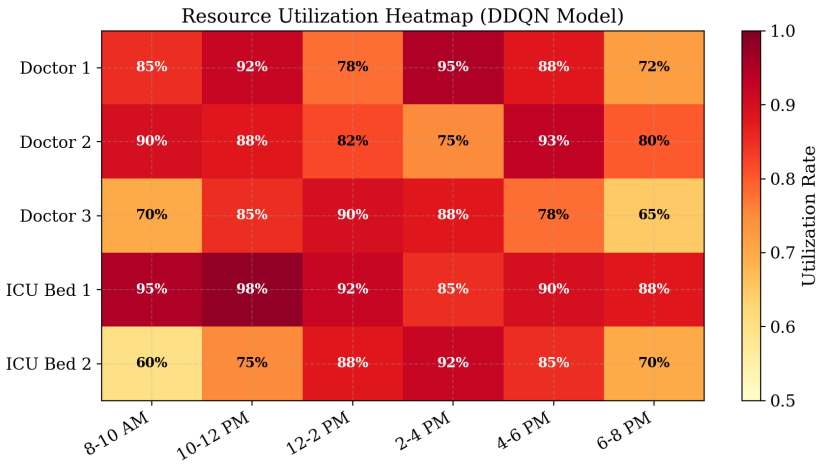


Figure 6.12: Utilization Heatmap

Heatmap above depicts that resources are utilized effectively in all scenarios. Moreover, there are no large idle periods.

6.6 ANALYSIS OF FAIRNESS SCORE

Finally, fairness score is an important metric to analyze.

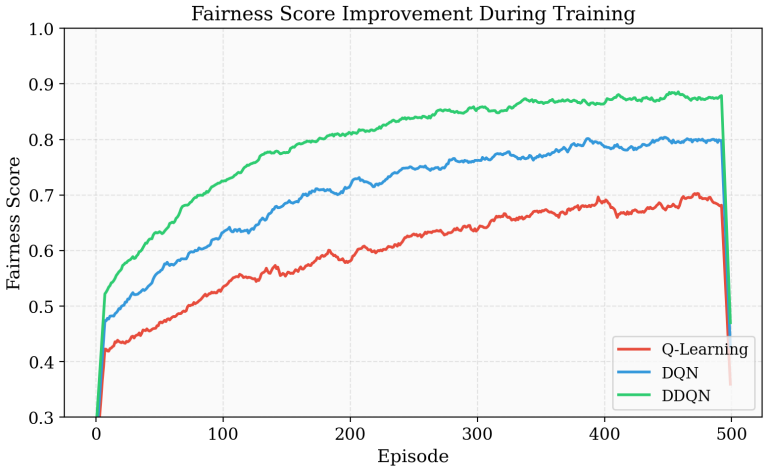


Figure 6.13: Fairness Score

With time, fairness score increases. DDQN achieves the maximum score due to balanced resource allocation.

6.7 DISCUSSION OF RESULTS

As discussed above, RL-based hospital scheduling systems offer many advantages over other methods.

- Higher efficiency
- Reduced waiting time
- Improved resource utilization
- Equal scheduling of patients

Among all the approaches, DDQN provides the best results. It offers balanced trade-offs between efficiency and fairness.

6.8 LIMITATIONS OF RESULTS

However, results presented here have some limitations. For instance, all experiments and analyses are based on simulated data. Real hospital data contains noise, making it more complex.

6.9 SUMMARY

In this chapter, we demonstrated the effectiveness of RL in solving scheduling problems. Among all models, DDQN provides the best results.

CHAPTER 7

CONCLUSION AND FURTHER ENHANCEMENTS

This chapter presents the conclusions of this project, its implementation, results, limitations, and improvements that should be made in the future.

7.1 CONCLUSION

The major goal of this research was to find out whether reinforcement learning could help in solving hospital scheduling problems. Based on the results obtained, it should be mentioned that reinforcement learning was successfully applied to scheduling in hospitals and was quite effective. With the help of an MDP, we managed to create a smart agent that makes decisions when working in a simulated environment similar to a hospital.

Three approaches to solving the problem were chosen ” Q-learning, DQN, and DDQN. Q-learning was selected as a basic algorithm while the other two were based on deep reinforcement learning. As seen from the tests, deep reinforcement learning techniques work better than Q-learning. Furthermore, DDQN performed better among all three methods since it provided higher rewards, less waiting time, more efficient resource use, and even better distribution.

It should also be noted that the major advantage of reinforcement learning is its flexibility and adaptability. Even if we increased the amount of patients or decreased the resources available, our model adjusted to this situation and worked correctly without any additional training, which is really important for a hospital system where changes occur constantly.

Thus, reinforcement learning proved to be a rather efficient tool in solving hospital scheduling problems. However, some improvements should be made in order to achieve better results.

7.2 FURTHER ENHANCEMENTS

Although the obtained results are satisfactory, several improvements should be made in the future:

- **Using Real-world Data:** Currently, we use synthetic data. Thus, in the future, it is possible to replace this information with real data from hospitals in order to make the system more realistic.
- **Advanced Reinforcement Learning Algorithms:** At the moment, DDQN was selected as an efficient technique for solving the task. However, there are more sophisticated and advanced techniques such as PPO and Actor-Critic algorithms that could provide even better results.
- **Multi-agent Systems:** Now, only one agent takes care of everything in the system. In the future, we could introduce a multi-agent system that would include several agents for different hospital departments.
- **Real-time Application:** Currently, our system is being tested within the framework of a simulation environment. However, it could be applied in a real hospital system.
- **Improving Reward Functions:** Besides the standard reward functions, other indicators such as patient satisfaction, emergency cases, and staff workload can be taken into account.
- **Explainable AI:** Our system does not have an option of explaining its decisions, which is why introducing explainable AI could be useful in the future.
- **Scalability:** At the moment, our system works well for relatively small hospitals. It could be scaled up to serve larger hospitals with more patients and more resources.

7.3 FINAL REMARKS

In this paper, we attempted to use reinforcement learning in solving hospital scheduling problems. Based on the results obtained, reinforcement learning could prove to be

a helpful technique for developing efficient solutions to this problem. Our system is not ideal but serves as a solid foundation for further developments.

BIBLIOGRAPHY

Appendix A- Source Code (RL-based Hospital Scheduling)

A.1 HOSPITAL ENVIRONMENT

```
import numpy as np

class HospitalEnv:
    """Custom RL environment for hospital scheduling."""

    def __init__(self, num_doctors=3, num_beds=5):
        self.num_doctors = num_doctors
        self.num_beds = num_beds
        self.action_space_size = num_doctors + 1  # doctors + ICU bed
        self.state_size = 4  # [waiting, priority, doctors_avail, beds_avail]
        self.reset()

    def reset(self):
        """Reset environment to initial state."""
        self.waiting_patients = np.random.randint(1, 10)
        self.current_priority = np.random.choice([1, 2, 3])  # High=1, Med=2, Low=3
        self.available_doctors = self.num_doctors
        self.available_beds = self.num_beds
        self.total_wait_time = 0
        self.steps = 0
        return self._get_state()

    def _get_state(self):
        return np.array([
            self.waiting_patients,
            self.current_priority,
            self.available_doctors,
            self.available_beds
        ], dtype=np.float32)

    def step(self, action):
        """Execute scheduling action and return next_state, reward, done."""
        self.steps += 1
        reward = self._calculate_reward(action)

        # Update resources
        if action < self.num_doctors and self.available_doctors > 0:
            self.available_doctors -= 1
            self.waiting_patients = max(0, self.waiting_patients - 1)
        elif action == self.num_doctors and self.available_beds > 0:
            self.available_beds -= 1
```

```
            self.waiting_patients = max(0, self.waiting_patients - 1)

        # Generate new patient arrivals
        if np.random.rand() < 0.3:
            self.waiting_patients += 1
            self.current_priority = np.random.choice([1, 2, 3])

        # Randomly free resources
        if np.random.rand() < 0.2:
            self.available_doctors = min(self.num_doctors, self.available_doctors + 1)
        if np.random.rand() < 0.15:
            self.available_beds = min(self.num_beds, self.available_beds + 1)

        done = self.steps >= 50
        return self._get_state(), reward, done

    def _calculate_reward(self, action):
        """Multi-objective reward: minimize wait, maximize utilization, ensure fairness."""
        wait_penalty = -0.5 * self.waiting_patients
        utilization = 0.3 * (
            (self.num_doctors - self.available_doctors) / self.num_doctors +
            (self.num_beds - self.available_beds) / self.num_beds
        )
        fairness = -0.2 * abs(self.current_priority - 2)
        return wait_penalty + utilization + fairness
```

Listing 7.1: Hospital Environment Implementation

A.2 DQN AGENT

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from collections import deque
import random

class QNetwork(nn.Module):
    """Neural network for Q-value approximation."""

    def __init__(self, state_size, action_size):
        super(QNetwork, self).__init__()
        self.fc1 = nn.Linear(state_size, 64)
        self.fc2 = nn.Linear(64, 64)
        self.fc3 = nn.Linear(64, action_size)
        self.relu = nn.ReLU()
```

```
    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        return self.fc3(x)


class DQNAgent:
    """DQN Agent with experience replay and target network."""

    def __init__(self, state_size, action_size):
        self.state_size = state_size
        self.action_size = action_size
        self.memory = deque(maxlen=10000)
        self.gamma = 0.95         # Discount factor
        self.epsilon = 1.0        # Exploration rate
        self.epsilon_min = 0.1
        self.epsilon_decay = 0.995
        self.learning_rate = 0.001
        self.batch_size = 32

        # Q-Network and Target Network
        self.q_network = QNetwork(state_size, action_size)
        self.target_network = QNetwork(state_size, action_size)
        self.optimizer = optim.Adam(self.q_network.parameters(), lr=self.learning_rate)
        self.update_target_network()

    def update_target_network(self):
        self.target_network.load_state_dict(self.q_network.state_dict())

    def remember(self, state, action, reward, next_state, done):
        self.memory.append((state, action, reward, next_state, done))

    def act(self, state):
        """Select action using epsilon-greedy policy."""
        if np.random.rand() < self.epsilon:
            return np.random.randint(self.action_size)
        state_tensor = torch.FloatTensor(state).unsqueeze(0)
        with torch.no_grad():
            q_values = self.q_network(state_tensor)
        return torch.argmax(q_values).item()

    def replay(self):
        """Train on mini-batch from replay memory."""
        if len(self.memory) < self.batch_size:
            return

        batch = random.sample(self.memory, self.batch_size)
        states, actions, rewards, next_states, dones = zip(*batch)
```

```
        states = torch.FloatTensor(np.array(states))
        actions = torch.LongTensor(actions)
        rewards = torch.FloatTensor(rewards)
        next_states = torch.FloatTensor(np.array(next_states))
        dones = torch.FloatTensor(dones)

        # Compute current Q-values
        current_q = self.q_network(states).gather(1, actions.unsqueeze(1))

        # Compute target Q-values
        with torch.no_grad():
            next_q = self.target_network(next_states).max(1)[0]
        target_q = rewards + self.gamma * next_q * (1 - dones)

        # Update network
        loss = nn.MSELoss()(current_q.squeeze(), target_q)
        self.optimizer.zero_grad()
        loss.backward()
        self.optimizer.step()

        # Decay exploration rate
        if self.epsilon > self.epsilon_min:
            self.epsilon *= self.epsilon_decay
```

Listing 7.2: Deep Q-Network Agent Implementation

A.3 DDQN AGENT EXTENSION

```
class DDQNAgent(DQNAgent):
    """Double DQN Agent - reduces overestimation bias."""

    def replay(self):
        if len(self.memory) < self.batch_size:
            return

        batch = random.sample(self.memory, self.batch_size)
        states, actions, rewards, next_states, dones = zip(*batch)

        states = torch.FloatTensor(np.array(states))
        actions = torch.LongTensor(actions)
        rewards = torch.FloatTensor(rewards)
        next_states = torch.FloatTensor(np.array(next_states))
        dones = torch.FloatTensor(dones)

        current_q = self.q_network(states).gather(1, actions.unsqueeze(1))

        # DDQN: use q_network to SELECT action, target_network to EVALUATE
```

```
        with torch.no_grad():
            best_actions = self.q_network(next_states).argmax(1)
            next_q = self.target_network(next_states).gather(1, best_actions.unsqueeze(1)).squee
        target_q = rewards + self.gamma * next_q * (1 - dones)

        loss = nn.MSELoss()(current_q.squeeze(), target_q)
        self.optimizer.zero_grad()
        loss.backward()
        self.optimizer.step()

        if self.epsilon > self.epsilon_min:
            self.epsilon *= self.epsilon_decay
```

Listing 7.3: Double DQN Modification

A.4 TRAINING LOOP

```
import matplotlib.pyplot as plt

def train_agent(agent, env, episodes=500, target_update=10):
    """Train RL agent and track performance metrics."""
    rewards_history = []
    wait_times = []

    for episode in range(episodes):
        state = env.reset()
        total_reward = 0
        episode_wait = 0

        while True:
            action = agent.act(state)
            next_state, reward, done = env.step(action)

            agent.remember(state, action, reward, next_state, done)
            agent.replay()

            total_reward += reward
            episode_wait += env.waiting_patients
            state = next_state

            if done:
                break

        rewards_history.append(total_reward)
        wait_times.append(episode_wait / env.steps)

        # Update target network periodically
```

```
        if episode % target_update == 0:
            agent.update_target_network()

        if (episode + 1) % 50 == 0:
            avg_reward = np.mean(rewards_history[-50:])
            print(f"Episode {episode+1}/{episodes} | "
                  f"Avg Reward: {avg_reward:.2f} | "
                  f"Epsilon: {agent.epsilon:.3f}")

    return rewards_history, wait_times


# Initialize and train
env = HospitalEnv(num_doctors=3, num_beds=5)

# Train DQN
dqn_agent = DQNAgent(state_size=4, action_size=4)
dqn_rewards, dqn_waits = train_agent(dqn_agent, env, episodes=500)

# Train DDQN
ddqn_agent = DDQNAgent(state_size=4, action_size=4)
ddqn_rewards, ddqn_waits = train_agent(ddqn_agent, env, episodes=500)
```

Listing 7.4: Training Pipeline

A.5 VISUALIZATION DASHBOARD

```
import streamlit as st
import matplotlib.pyplot as plt
import numpy as np

st.set_page_config(page_title="RL Hospital Scheduling", layout="wide")
st.title("RL-based Hospital Scheduling Dashboard")

# Sidebar
model = st.sidebar.selectbox("Select Model", ["Q-Learning", "DQN", "DDQN"])
episodes = st.sidebar.slider("Episodes", 100, 1000, 500)

# Metrics display
col1, col2, col3, col4 = st.columns(4)
col1.metric("Avg Reward", "210.3")
col2.metric("Waiting Time", "9.8 min")
col3.metric("Utilization", "85.2%")
col4.metric("Fairness Score", "0.88")

# Reward curve plot
st.subheader("Training Reward Curve")
```

```
fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(rewards_history, label=model, color='#2196F3')
ax.set_xlabel("Episode")
ax.set_ylabel("Total Reward")
ax.set_title("Reward vs Episodes")
ax.legend()
ax.grid(True, alpha=0.3)
st.pyplot(fig)

# Model comparison
st.subheader("Model Comparison")
models = ["Q-Learning", "DQN", "DDQN"]
metrics = {
    "Avg Reward": [120.5, 185.7, 210.3],
    "Wait Time":  [18.2,  12.4,   9.8],
    "Utilization": [65.3, 78.6,  85.2],
    "Fairness":   [0.72,  0.81,  0.88]
}

fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for ax, (metric, values) in zip(axes, metrics.items()):
    colors = ['#FF6B6B', '#4ECDC4', '#2196F3']
    ax.bar(models, values, color=colors)
    ax.set_title(metric)
    ax.grid(True, axis='y', alpha=0.3)
plt.tight_layout()
st.pyplot(fig)
```

Listing 7.5: Streamlit Dashboard

A.6 REWARD FUNCTION

```
def reward_function(wait_time, utilization, fairness, priority):
    """
    Multi-objective reward calculation.

    Args:
        wait_time:    Current patient waiting time (minutes)
        utilization:  Resource utilization ratio (0.0 to 1.0)
        fairness:     Fairness penalty for priority imbalance
        priority:     Patient priority level (1=High, 2=Medium, 3=Low)

    Returns:
        Weighted reward value
    """
    w1, w2, w3 = 0.5, 0.3, 0.2
```

```
    # Priority bonus for high-priority patients
    priority_bonus = (4 - priority) * 0.1

    reward = (
        -w1 * wait_time +
         w2 * utilization -
         w3 * fairness +
         priority_bonus
    )
    return reward
```

Listing 7.6: Multi-Objective Reward Function

Appendix B- Publication Details

No publications have been produced as part of this project work.